

Step 2: Data Enrichment and Preprocessing (No AI/ML)

Executive Summary. After raw telemetry is collected (logs, metrics, traces, events), a deterministic enrichment/preprocessing pipeline adds contextual metadata before storage. Dynatrace's **OneAgent** and operator auto-instrumentation populate key fields (service names, host/OS details, container/Kubernetes labels, trace/span IDs, cloud info, etc.) at the source. OpenTelemetry instrumentation likewise carries resource attributes (e.g. `service.name`, `cloud.account.id`) that Dynatrace automatically maps to its semantic model [14†L19-L27] [27†L63-L71]. In a custom pipeline you would ingest data, parse and normalize it (e.g. convert log text to JSON fields), then enrich each record by merging in host/container/process metadata (from files, APIs or env), deduplicate if needed, index for search, and finally store. All of these steps use fixed rules or code (no ML). Later stages (Step 3) may apply ML for anomaly detection, but enrichment is rule-based. Essential components include collectors or agents to capture data, processors/detectors to extract metadata, and storage back-ends (e.g. OpenSearch, ClickHouse, etc). Security controls (TLS, RBAC) govern access. Importantly, privacy is handled via **masking** or **hashing** sensitive fields: e.g. OneAgent can mask PII in logs at capture [48†L25-L32], and ingest pipelines can apply regex or hashing rules [47†L34-L43] [48†L25-L32].

Below is a detailed breakdown of metadata sources, enrichment mechanisms, processing stages, and implementation guidance, with examples and diagrams.

Key Telemetry Metadata and Sources

- **Service Name and Tags.** Dynatrace names services from telemetry: it uses OTEL resource attributes like `service.name` or `k8s.workload.name` to label services [14†L19-L27]. For example, setting `OTEL_SERVICE_NAME=my-checkout-service` on an app will cause Dynatrace to assign that name to its spans/metrics [14†L63-L72]. OneAgent-instrumented processes also detect framework context or use `OTEL_RESOURCE_ATTRIBUTES` for naming [14†L79-L86]. Additional context (environment, region, team) should be captured as *tags* (fields) rather than conflated into the name [14†L88-L97].
- **Host and Process Metadata.** OneAgent (or any system collector) gathers host-level details: hostname, OS type/version, CPU/Memory specs, host tags (environment, datacenter, role). These often come from reading local files or the OS (e.g. `/etc/os-release`, hostname, system APIs). OneAgent writes host metadata into virtual files (e.g. `/var/lib/dynatrace/enrichment/dt_host_metadata.json`) [49†L133-L142]. Example fields include `host.name`, `host.os.name`, `host.os.version`, and custom tags (e.g. `primary_tags.environment`). Process information (PID, executable path, process group) also is collected; OTEL spec defines attributes like `process.pid`, `process.command`, `process.runtime.name` for this. If no agent is available, a collector can run on the host (as an agent or DaemonSet) and use system probes (e.g. Python's `psutil`) to fetch similar info.
- **Container and Kubernetes Metadata.** For containerized workloads, we collect container-specific metadata and pod/cluster context. OpenTelemetry has **Resource Detectors** and a *k8sattributes* processor that query the Kubernetes API or environment to pull data. For example,

Dynatrace shows a config using the `k8sattributes` processor to extract `k8s.pod.name`, `k8s.namespace.name`, `k8s.node.name`, `k8s.container.name`, plus higher-level controller names (deployment, replicaset, etc.) [17†L169-L177]. Likewise, Dynatrace's metadata enrichment (via its Operator) injects a JSON file (`dt_metadata.json`) with fields like `k8s.cluster.name`, `k8s.namespace.name`, `k8s.pod.name`, `k8s.container.name`, `k8s.node.name`, etc. [30†L136-L144] [30†L166-L174]. These attributes travel with every log/span/metric, enabling filters by Kubernetes labels, namespace, node, etc. To obtain them in a custom pipeline, you can:

- Use the **Downward API** or environment variables in pods (e.g. injecting `POD_NAME`, `POD_NAMESPACE` from `metadata.name` / `metadata.namespace`).
- Use a library (e.g. `kubernetes-client` in Python) to call the K8s API given pod name/namespace.
- Use OpenTelemetry's `k8sattributes` processor or similar (as shown in Dynatrace's docs [17†L169-L177]).
- Parse the Operator's `dt_metadata.json` if OneAgent/Operator is present.
- **Trace and Correlation IDs.** Spans from distributed tracing carry a `trace_id` and `span_id`. This is **already an enrichment**: it links related requests. OneAgent's distributed tracing (or any OTEL instrumentation) ensures every span includes its trace context and `service.name`. For logs, one can embed trace IDs (if the application logs them) or correlate by timestamp. OTEL's semantic conventions (and Dynatrace mapping) also define `span.kind`, `trace_state`, etc. Dynatrace's pipeline can join logs with traces via shared attributes (e.g. `dt.trace.id`). This linking requires that the instrumentation library propagates context (in code or via headers), which is standard in OTEL and OneAgent.
- **Cloud and Environment Attributes.** If running in a cloud (AWS, Azure, GCP), one can query the cloud **Instance Metadata Service (IMDS)** to fetch attributes. For example, AWS EC2 metadata at `http://169.254.169.254/latest/meta-data/` can yield the instance ID, region, tags, etc. In OpenTelemetry, the *resource detection processor* for AWS EC2 uses the EC2 metadata API (and optionally the DescribeTags API) to set attributes like `aws.account.id`, `aws.region`, `k8s.cluster.name` (for EKS) [38†L529-L538]. Dynatrace then maps generic OTEL attributes to provider-specific fields: e.g. `cloud.account.id` → `aws.account.id` (or `azure.subscription`, `gcp.project.id`) and `cloud.region` → `aws.region` / `azure.location` [27†L68-L77]. In practice, one might use libraries like `boto3` (AWS), `azure-mgmt` (Azure), or `google-cloud` to call metadata endpoints. For example, the OTEL resource detector notes: "Uses the EC2 instance metadata API to retrieve related resource attributes... tags can be fetched via IMDS (no IAM needed) or the EC2 DescribeTags API (requires IAM)." [38†L529-L538]. Similar detectors exist for Azure ("Azure Instance Metadata Service") and GCP. Even if not using OTEL, your pipeline can run a step that queries these endpoints (with Python's `requests`) and adds fields like `cloud.provider=aws`, `aws.region=us-east-2`, `aws.account.id=123456789012`.

Instrumentation and Extraction Mechanisms

Telemetry enrichment uses several mechanisms:

- **Agent Auto-Instrumentation.** Dynatrace OneAgent (for Java, .NET, Node.js, etc.) auto-instruments apps to capture traces, metrics, and logs. It **inlines code** into processes or hooks

into common libraries to record spans and metrics without code changes. OneAgent also reads environment settings (e.g. `OTEL_RESOURCE_ATTRIBUTES`, `DT_TAGS`) and automatically adds those attributes to spans and logs. For example, setting `OTEL_SERVICE_NAME` on the process sets the service name seen across telemetry [14†L63-L72]. OneAgent also reads host/container metadata from OS interfaces (procfs, cgroups) and its enrichment files.

- **OpenTelemetry SDKs and Conventions.** If using OTEL libraries (e.g. the OpenTelemetry Java/Python SDKs), the code can set **resource attributes**. The standard conventions (in [OTEL Spec](#)) include keys like `service.name`, `process.pid`, `os.type`, `host.name`, `container.id`, `k8s.pod.name`, `cloud.provider` etc. Any attributes set here are captured by the collector or OneAgent's OTLP exporter. For example, OTEL's `resource_detection` processor can populate fields like `container.id` (by inspecting `/proc` or Docker API), or the `env` detector reads `OTEL_RESOURCE_ATTRIBUTES`. Dynatrace will map all standard OTEL attributes to its internal fields (see "Semantic Dictionary"). Use of these conventions means minimal extra work: just configure your SDK. (Dynatrace supports the full [OTEL semantic mapping](#); e.g. `cloud.account.id` becomes `aws.account.id` or `azure.subscription` [27†L68-L77].)
- **Environment Variables.** You can set **environment variables** in your app/container to tag all telemetry. Common patterns:
 - `OTEL_RESOURCE_ATTRIBUTES`: e.g. `service.name=myservice, deployment.environment=prod`.
 - `OTEL_SERVICE_NAME` (legacy) just sets `service.name`.
 - `DT_TAGS` (Dynatrace-specific tag injection). These propagate into spans, metrics, and logs. Dynatrace's docs note that setting `OTEL_RESOURCE_ATTRIBUTES=deployment.release_version='v1'` will automatically enrich traces and logs with that metadata [12†L171-L180].
- **Kubernetes API / Downward API.** Inside a pod, environment variables can expose Kubernetes context (e.g. `POD_NAME`, `POD_NAMESPACE`). OpenTelemetry's `k8sattributes` processor uses this to call the K8s API and fetch pod labels, node, etc. Similarly, tools can mount the ServiceAccount and use the K8s API (with RBAC permissions) to enrich telemetry. Dynatrace's example configuration shows enabling a K8s service account with `get/watch` on pods and namespaces so the collector can fetch pod metadata [17†L289-L297] [17†L415-L424].
- **cgroup/Procfs & Container Runtime.** For pure container metadata, one can inspect `/proc/<pid>/cgroup` to find the container ID, or use the Docker (or container runtime) API/CRI socket. The OpenTelemetry Docker resource detector, for instance, queries the Docker daemon via its socket to get container labels and name [33†L360-L369]. In Kubernetes, if a pod has multiple containers, `k8s.container.name` is only extracted if only one container is present or a container ID is included in the signal [17†L419-L427].
- **Cloud Metadata Endpoints.** As above, cloud VMs provide HTTP metadata endpoints (AWS, Azure, GCP, etc.). These can be queried (often at `169.254.169.254` in AWS; `http://169.254.169.254/metadata/instance` on Azure, `http://metadata.google.internal` on GCP) to fetch instance ID, region, instance tags, etc. The OTEL AWS detector (with `tags_from_imds`) can do this without IAM keys [38†L536-L544]. After obtaining values (e.g.

`instance-id` or `zone`), add them as telemetry attributes. Dynatrace will label them accordingly (`aws.instance.id` , `gcp.zone` , etc.) per its mapping [\[27†L68-L77\]](#) .

Processing Pipeline Stages

The typical enrichment pipeline follows these sequential stages:

1. **Ingest.** Raw data enters the pipeline. Logs may be collected by a log agent (Fluent Bit/Fluentd/Vector/OneAgent Log Module) and sent (via HTTP or Kafka) to the pipeline. Metrics and traces are pushed by OTLP exporters (from OneAgent or OTEL SDKs) to a collector or gateway. Ensure all ingestion points use the same schema (e.g. JSON or OTLP protobuf).
2. **Parse and Normalize.** Unstructured logs are parsed into fields (e.g. using Grok or regex) so that each log message becomes a JSON object with well-known keys (`timestamp` , `message` , `level` , etc.). Metrics may be tagged with metadata (host, pod) and given standard field names. Distributed traces are often already structured, but you may translate attributes (e.g. replace `http.status_code` with Dynatrace's `http.response.status_code`). In Dynatrace's OpenPipeline, the **Processing** stage "parses values into fields, transforms the schema, and filters the data records. Fields are edited, and sensitive data is masked" [\[55†L64-L68\]](#) . Custom pipelines let you "Rename fields", "Add fields", or "Drop records" as needed [\[55†L66-L74\]](#) .
3. **Enrich.** Merge in contextual metadata. This may occur partly *before* sending to the pipeline (as with OneAgent embedding tags), and partly *in* the pipeline. Key steps:
4. **Add Host/Container Data:** Look up the host or container identifiers. For example, read `/var/lib/dynatrace/enrichment/dt_metadata.json` and incorporate its key/value pairs into the record (Python example in [\[49\]](#) shows reading this JSON into a dict). Or use an OTEL **resource detector**/processor that attaches host/container attributes from the running environment (system detector, Docker detector, K8s detector, AWS detector, etc.) [\[38†L529-L538\]](#) .
5. **Add Kubernetes Data:** If the record originated from a Kubernetes pod, add `k8s.cluster.name` , `k8s.namespace.name` , `k8s.pod.name` , etc. This can be done by querying the K8s API (via an OTEL `k8sattributes` processor or custom code using `kubernetes-client`), or by reading the Operator's injected metadata file. Dynatrace's Kubernetes enrichment config (above) shows this extraction in action [\[17†L169-L177\]](#) .
6. **Add Cloud Data:** Query cloud metadata endpoints or resource detectors to set `cloud.provider` , `cloud.region` , `aws.account.id` , `azure.subscription` , etc. Then map those to Dynatrace fields (e.g. OTEL `cloud.account.id` → `aws.account.id` [\[27†L68-L72\]](#)).
7. **Add Service/Process Tags:** Ensure the service name (`service.name`) is on every trace/span/log. If needed, read it from OTEL attributes or environment variables. You might also tag records with process group IDs or custom tags (e.g. `primary_tags.environment`) using a lookup or config.
8. **Parse Semantic Fields:** Dynatrace will parse semantic attributes (e.g. splitting `url.full` into `url.path` , `server.address` , etc.) automatically [\[27†L38-L46\]](#) . In a custom pipeline, you might similarly decompose complex fields or extract structured data (e.g. parse JSON or CSV).
9. **Deduplicate.** Identify and drop duplicate records to save storage. For logs, this may involve filtering identical log lines within a short window. In Dynatrace's OpenPipeline, you could use a

“Drop record” processor for duplicates or use hashed keys. For metrics/spans, unique IDs naturally prevent duplicates. (Dynatrace’s docs hint that data “remains in its original, structured form... ensures no information is lost” [55†L208-L217] , but duplicate suppression can still be applied if needed.)

10. **Index.** Fields of each record should be indexed or added to searchable schema. In Dynatrace/Grail, top-level fields become queryable dimensions; in a custom system, you might send logs to an index (Elasticsearch/OpenSearch) or label metrics accordingly. For example, mark `service.name` , `k8s.namespace.name` , and `host.name` as indexed tags.
11. **Store.** Finally, write the enriched records to the backend storage. This could be a time-series database (for metrics), a document store (for logs), or a tracing DB. For scale, often a streaming bus (Kafka) is used between stages, with consumers writing to multiple stores (e.g. one topic/consumer group for logs to OpenSearch, one for metrics to Prometheus/ClickHouse, etc.). Dynatrace uses Grail (a scalable data lake) under the hood, but a DIY solution might use e.g. OpenSearch for logs/events, ClickHouse or a TSDB for metrics, and MongoDB or PostgreSQL for generic events.

Processing Pipeline Checklist

To implement the above pipeline, ensure you have:

- **Receivers/Collectors:** Tools to pull data in. For logs: Fluent Bit/Fluentd/Vector or the OneAgent Log Module. For metrics: OpenTelemetry Collector or Prometheus exporters. For traces: OTLP/gRPC or HTTP receivers.
- **Parser Normalizers:** Regex or Grok rules for each log format. OTEL processors or custom code to unify metric and trace schemas.
- **Enrichment Processors:**
 - OTEL *resource detectors* (`env` , `system` , `docker` , `k8s` , `aws` , `azure` , etc.) configured as needed [33†L283-L292] [38†L529-L538] .
 - Kubernetes Attributes processor (for pod/cluster data) [17†L169-L177] .
 - Transform or attribute processors to rename/merge keys.
 - Custom scripts or processors to read metadata files (e.g. Dynatrace’s `dt_metadata.json`) and apply as resource attributes.
- **Deduplication Steps:** e.g. hashed sets or pipelines with state to drop repeats.
- **Exporters:** Components to send data to the next stage or storage (Kafka producers, database clients, or OTLP exporters to a backend).
- **RBAC/Permissions:**
 - If using K8s APIs: a service account with `get/watch/list` on pods/namespaces (as in [17] lines 321-330).
 - If querying cloud APIs: ensure IAM roles/policies allow `EC2:DescribeTags` or IMDS access, and that IMDS tags are enabled if using the EC2 metadata method [38†L536-L544] .
 - If Docker detection: mount `/var/run/docker.sock` with proper permissions [33†L360-L369] .
- **Libraries/Tools:**
 - For host info: `psutil` (Python) or similar for CPU/memory.
 - For Kubernetes: `kubernetes-client` or equivalent to call the K8s API.
 - For AWS: `boto3` (Python) or `aws-sdk` to call Instance Metadata or DescribeTags.
 - For Azure/GCP: `requests` to metadata endpoints (Azure IMDS requires a special header, GCP metadata needs `Metadata-Flavor:Google`).

- For data parsing: regex library, `json`, etc.

Data Models and Examples

Each enriched record should follow a consistent JSON schema. For example:

```
// Enriched log example
{
  "timestamp": "2026-06-23T12:34:56.789Z",
  "service.name": "user-auth-service",
  "host.name": "web-01",
  "k8s.cluster.name": "cluster-east",
  "k8s.namespace.name": "prod",
  "k8s.pod.name": "auth-pod-abc123",
  "k8s.container.name": "auth-container",
  "container.id": "0a1b2c3d4e5f",
  "process.id": 4321,
  "cloud.provider": "aws",
  "aws.account.id": "123456789012",
  "aws.region": "us-east-2",
  "message": "User login successful",
  "log.level": "INFO"
}
```

```
// Enriched metric example
{
  "timestamp": "2026-06-23T12:34:00.000Z",
  "metric.name": "cpu.utilization",
  "value": 75.3,
  "unit": "percent",
  "service.name": "user-auth-service",
  "host.name": "web-01",
  "k8s.namespace.name": "prod",
  "k8s.pod.name": "auth-pod-abc123",
  "aws.region": "us-east-2"
}
```

```
// Enriched trace/span example
{
  "trace_id": "4c1f...e2a6",
  "span_id": "9f4b...7d3c",
  "parent_span_id": "empty if root",
  "service.name": "user-auth-service",
  "name": "HTTP GET /login",
  "attributes": {
    "http.method": "GET",
    "http.status_code": 200
  },
}
```

```

"resource": {
  "host.name": "web-01",
  "k8s.pod.name": "auth-pod-abc123",
  "k8s.namespace.name": "prod",
  "aws.account.id": "123456789012",
  "aws.region": "us-east-2",
  "process.id": 4321
}
}

```

These JSON records combine application data with the enriched fields. In practice, your pipeline might flatten or nest attributes differently.

Enrichment Methods Comparison

Method	Ease (Setup)	Coverage (Data Sources)	Performance Overhead	Security/Permissions	Control/Customization
OneAgent (Agent)	Very easy (install and auto-deploy)	Very broad (host, OS, apps, network, all processes) [14†L19-L27]	Moderate (native hooking adds overhead to each app) [45†L315-L323]	Requires high privileges (root/container runtime access)	Lower (proprietary, limited config beyond env vars)
OTel Collector (Gateway)	Moderate (deploy central service)	Broad (accepts OTLP from any app, plus logs via Fluent)	Scales well (batch processing offloads apps) [45†L315-L323]	Needs secure endpoints/TLS; runs with broad network access	High (flexible processors, transforms, multi-backend) [45†L313-L322]
Sidecar (OTel Agent)	Easy per-pod (DaemonSet or injection)	App-level only (captures that pod's telemetry)	Very low overhead (local, minimal processing) [45†L315-L323]	Runs in pod's user context (limited host info)	Medium (limited processors, simpler config)
SDK Instrumentation	Developer effort needed	Limited (only instrumented code paths)	Varies (adds code, but minimal if done well)	No extra privileges	High (code-level control, can add any field)

Sources: The CNCF OpenTelemetry blog notes that a Collector “offloads heavy processing away from apps” whereas an agent is “very lightweight” [\[45†L315-L323\]](#) . In other words, agents are easiest to

run but cover only local data (and OneAgent specifically requires elevated access), while a centralized collector can process many data sources at once with more flexibility [45†L315-L323] .

Topology and Trace Linking

To build service maps and link telemetry:

- **Trace IDs/Span IDs:** Use OTEL or OneAgent tracing to propagate a `trace_id` through call chains. Dynatrace's distributed tracing uses these to link spans across services automatically. Each span carries the service name (as above) so that the trace can be resolved to a chain of services.
- **Service Tags:** Besides names, add tags like `k8s.namespace`, `primary_tags.team`, etc. These allow filtering service calls by environment or team. As [14] advises, keep names simple and use tags for context [14†L88-L97] .
- **Process-to-Container Mapping:** Determine which container (or host) a process belongs to. One way is to read the process's cgroup: e.g., in Python:

```
pid = os.getpid()
with open(f"/proc/{pid}/cgroup") as cg:
    for line in cg:
        if "docker" in line or "kubepods" in line:
            container_id = line.split("/")[-1].strip()
            break
```

Then use that container ID to attach `k8s.container.name` or `container.id`. OpenTelemetry's Docker detector similarly mounts the Docker socket to find attributes [33†L360-L369] .

- **Port/Protocol Inference:** If logs or metrics include networking info (e.g. a connection to port 5432), you might infer service (PostgreSQL) or protocol. This is typically done by the instrumentation library (e.g. HTTP client spans include `http.url`, `server.port`). Dynatrace can auto-parse URLs (splitting `url.full` into components) [27†L38-L46] . If building your own, use regex or parsers: e.g. extract IP:Port and map it to known services via a config table.
- **Smartscape/Entity Graph:** Dynatrace builds a topology graph (Smartscape) by associating logs/spans with entities (services, processes, containers, hosts). In a custom solution, you can link entities by shared attributes: e.g., log records that contain a process ID can be mapped to the corresponding service entity; spans link services via calls. Dynatrace's OpenPipeline "Smartscape Node Extraction" explicitly attaches Smartscape nodes to logs/events [55†L95-L103] , which you can mimic by tagging each record with an entity ID if known.

Performance, Scalability, and Storage

High-volume data demands scalable design:

- **Streaming/Pub-Sub:** Use a message broker (Kafka, Pulsar, etc.) to buffer data between pipeline stages. This decouples producers (collectors) from consumers (indexers, storage) and provides

durability. It also allows horizontal scaling of pipeline components (e.g. multiple parser/enricher consumers).

- **Batching and Compression:** Where possible (especially for metrics and logs), batch records. OTLP exporters and Fluent Bit support batching and gzip/deflate compression to reduce network overhead. Enrichment steps (like querying metadata) should be cached or done infrequently to avoid per-record cost.
- **Indexing vs. Columnar Storage:**
 - Use **Elasticsearch/OpenSearch** or **ClickHouse** for logs and queryable events. Elasticsearch is easy to integrate but can be expensive at scale; ClickHouse is a high-performance columnar store (used by many large observability systems) that supports *sub-second queries on petabytes of data* [57†L565-L573] . For example, Datadog notes ClickHouse “supports sub-second analysis across petabytes of logs, metrics, traces, and events” [57†L565-L573] , making it suitable for high-cardinality telemetry.
 - For metrics, consider a time-series DB (Prometheus, InfluxDB) or even ClickHouse’s Prometheus integration. If using Grafana/Prometheus, push metrics to a remote storage adapter (some use ClickHouse or Thanos).
 - For relational data (e.g. process/entity metadata), a database like PostgreSQL or MongoDB can store lookup tables (e.g. host inventory), but raw telemetry often needs specialized stores (time-series or log indexes).
- **Scaling Enrichers:** The components that do enrichment (collectors, processors) should themselves be scalable. Use Kubernetes for horizontal scaling of collectors/agents. Ensure they have enough memory/CPU for the processing load (regex parsing, API calls). Use autoscaling where possible.

Security, Privacy, and PII Handling

- **Data in Transit:** Encrypt all telemetry in motion (use TLS for OTLP/HTTP, secure Kafka with SASL/SSL). Apply strict authentication (API tokens) to prevent data injection or interception.
- **Masking/Redaction:** Remove or hash sensitive information as early as possible. Dynatrace offers two modes: **at-capture** and **at-ingest** [47†L27-L35] . For example, OneAgent can be configured with masking rules so that PII (emails, credit card numbers, SSNs, IPs, etc.) is replaced before leaving the host [48†L25-L32] . Alternatively, configure your log agent (Fluent Bit, OpenTelemetry Collector) to drop or mask patterns (via regex or processors) before sending. As a second layer, use ingest-time processors: e.g. in OpenPipeline you might use a DQL or transform rule to `mask` or `hash` fields. The key is to ensure that sensitive strings never end up in the final store unless anonymized. Dynatrace specifically notes that masking on OneAgent “ensures sensitive data never leaves your environment” [48†L25-L32] .
- **Privacy Settings:** Only send metadata that’s necessary. Do not enrich with personal data unless required. Use one-way hashing if you need to correlate on an ID without storing the raw value. For example, replace user IDs or emails with SHA-256 hashes (Dynatrace masking rules can do SHA-256) [48†L61-L69] .

- **Access Control:** Limit who can configure or query the pipeline. Use RBAC in Kubernetes, and audit logs from the backend. When calling cloud metadata APIs, minimize permissions (e.g. prefer IMDS tags over full API calls so you don't need broad IAM rights [【38†L537-L545】](#)). Consider network policies: restrict who can reach the metadata endpoints or the collector ports.

AI/ML in Enrichment

No AI/ML is used in these enrichment steps. All parsing, normalization, and attribute addition is based on fixed rules, code, or semantic conventions. Machine learning is not needed (and generally not useful) for extracting obvious metadata like host name or container ID. Any “learning” about topology or classification happens later (e.g. in anomaly detection, root cause analysis).

Where ML might come later: After data is enriched, you might apply unsupervised learning to cluster logs or detect anomalies, or supervised models to classify events. But for the enrichment pipeline itself, ML is not recommended. The rules-driven approach is more reliable and transparent: e.g. regex or config for masking, explicit mappings for fields.

If you did explore ML for enrichment tasks, it would likely be unsupervised. For example, using NLP or clustering to identify new entity names or to auto-tag logs by topic. That requires a large corpus of labeled logs for training (supervised methods) or sophisticated unsupervised algorithms plus human review. In practice, it's easier to manually map attribute names or write one-off code. Therefore, **supervised ML is not needed** for basic enrichment; you won't gain much from trying to “learn” container names or host IDs, which are already well-defined by the environment.

Implementation Plan (Custom Pipeline)

1. **Data Collection:** Deploy agents/collectors:
2. **Logs:** Use Fluent Bit (as DaemonSet) or OneAgent to tail log files. Configure it to forward logs (e.g. via HTTP to a collector or Kafka). If using OpenTelemetry, use the OTLP exporter on your apps or Fluentd to send to an OTel collector.
3. **Metrics:** Use Prometheus exporters or the OTel Collector's host metrics receiver. If OneAgent is available, it will push host metrics via ActiveGate.
4. **Traces:** Instrument apps with OTel SDKs (auto-instrument or manual) or OneAgent. Configure them to send to an OTLP receiver.
5. **Collector Setup:** Run an **OpenTelemetry Collector** (or Dynatrace OTel distribution) in agent mode (as sidecar) or as a gateway. Include these components:
6. Receivers (OTLP/gRPC, HTTP, Zipkin, etc).
7. **ResourceDetectionProcessor:** Enable detectors for your environment (e.g. `aws`, `azure`, `gcp`, `k8s`).
8. **KubernetesAttributes Processor:** Extract pod and cluster data as shown in Dynatrace's example [【17†L169-L177】](#) .
9. **Transform Processor:** For adding static attributes (e.g. set `k8s.cluster.name` from an env var) or for OTEL-to-Dynatrace mapping as needed [【16†L82-L91】](#) .
10. **Batch, Retry, Timeout:** To optimize throughput and reliability.

11. Exporters: Send metrics/traces to Dynatrace (ActiveGate/OTLP endpoint) or to your storage (e.g. Prometheus remote write, Jaeger, etc).
12. **Log Parsing/Enrichment:** For logs in Fluent Bit or Vector:
13. Use parsers (Grok, regex) to extract fields like timestamp, severity, message.
14. Use **filter plugins/processors** to add metadata:
 - A *file-include* or *Lua* filter to read Dynatrace `dt_metadata.json` and merge into each log record.
 - A Kubernetes filter (builtin in Fluentd/FluentBit) to add `kubernetes.*` fields.
 - A *record_modifier* to add static tags (e.g. `environment=prod`) from env vars.
15. Apply **masking filters**: e.g. Fluent Bit's `modify_mask` or regex to redact emails/PII.
16. Forward enriched logs to Kafka or directly to Dynatrace.
17. **Deduplication:** Implement dedupe as needed. For example, Fluent Bit has a `record_modifier` with skip duplicates, or you can use a stateful processor (Redis-based) to track seen IDs. Only keep unique events.
18. **Storage:** Connect the final outputs:
19. For logs: e.g. an OpenSearch or Elasticsearch cluster, or push through ActiveGate to Dynatrace's Grail.
20. For metrics: remote write to Prometheus-compatible storage (ClickHouse, VictoriaMetrics) or Dynatrace.
21. For traces: Jaeger backend or Dynatrace's tracing.
22. **Monitoring and Scaling:**
23. Monitor pipeline throughput and latency.
24. Scale collectors horizontally.
25. Partition Kafka topics if used.

Code Snippets

Metadata extraction examples (Python pseudocode):

```
import json, os

# Example: Load Dynatrace OneAgent enrichment JSON files
enrich_attrs = {}
paths = [
    "/var/lib/dynatrace/enrichment/dt_metadata.json",
    "/var/lib/dynatrace/enrichment/dt_host_metadata.json"
]
for path in paths:
```

```

try:
    with open(path) as f:
        data = json.load(f)
        enrich_attrs.update(data)
except FileNotFoundError:
    pass

# Now enrich_attrs contains keys like 'k8s.namespace.name', 'k8s.pod.name',
# etc.
# Attach these to your telemetry records.

```

```

import requests

# Example: Query AWS metadata for region and instance ID
try:
    token = requests.put("http://169.254.169.254/latest/api/token",
headers={"X-aws-ec2-metadata-token-ttl-seconds": "30"}).text
    headers = {"X-aws-ec2-metadata-token": token}
    instance_id = requests.get("http://169.254.169.254/latest/meta-data/
instance-id", headers=headers).text
    region = requests.get("http://169.254.169.254/latest/meta-data/placement/
region", headers=headers).text
    enrich_attrs["aws.instance.id"] = instance_id
    enrich_attrs["aws.region"] = region
except Exception:
    # handle exceptions (e.g. not on EC2)
    pass

```

```

from kubernetes import client, config

# Example: Fetch Kubernetes pod labels for enrichment
config.load_incluster_config()
v1 = client.CoreV1Api()
pod_name = os.environ["POD_NAME"]
namespace = os.environ["POD_NAMESPACE"]
try:
    pod = v1.read_namespaced_pod(name=pod_name, namespace=namespace)
    enrich_attrs["k8s.labels"] = pod.metadata.labels
    enrich_attrs["k8s.annotations"] = pod.metadata.annotations
except client.ApiException:
    pass

```

```

# Example: Correlating traces/logs by trace_id (assuming OTLP spans)
trace_id = span.context.trace_id
span_id = span.context.span_id
# Later, attach trace_id to logs or use for filtering.

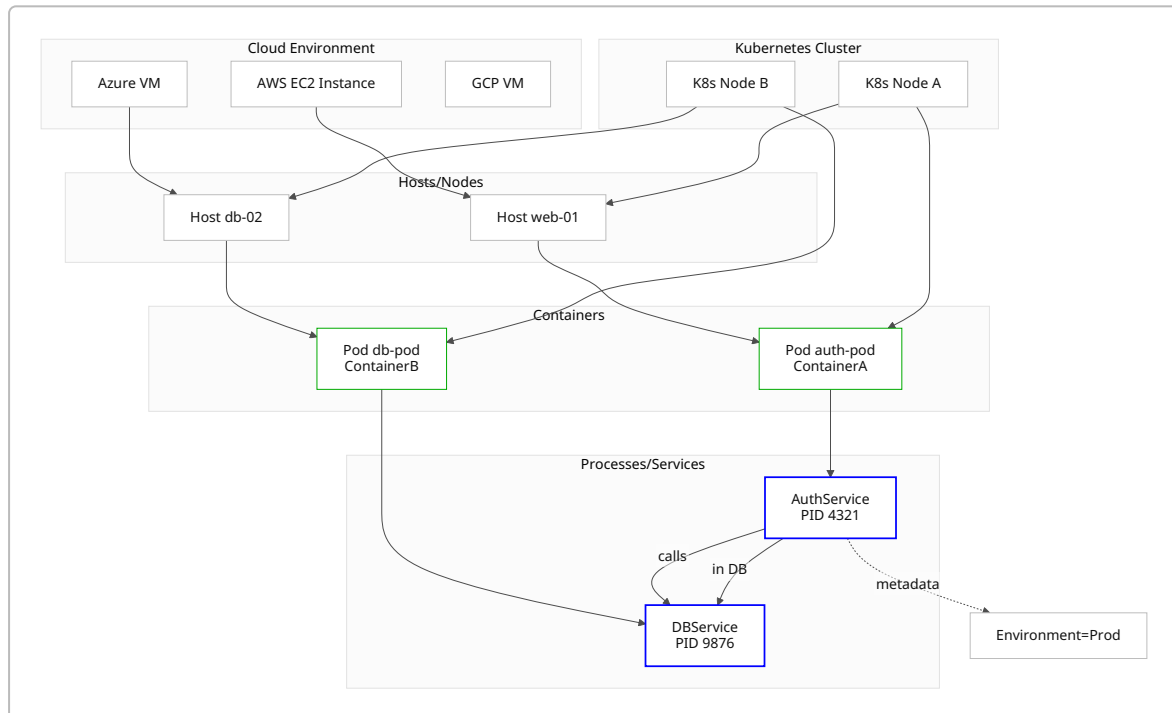
```

Diagrams

Processing Pipeline (flowchart):

```
flowchart LR
    A[Applications/Servers] --> B[Ingest Collectors<br/>(Fluent Bit, OTEL, OneAgent)]
    B --> C[Parse/Normalize]
    C --> D[Enrich Metadata]
    D --> E[Dedupe / Filter]
    E --> F[Indexing]
    F --> G[Storage/Analytics Backends]
    style A fill:#f9f,stroke:#333,stroke-width:1px
    style B fill:#bbf,stroke:#333,stroke-width:1px
    style C fill:#bbf,stroke:#333,stroke-width:1px
    style D fill:#cfc,stroke:#333,stroke-width:1px
    style E fill:#fdd,stroke:#333,stroke-width:1px
    style F fill:#fcf,stroke:#333,stroke-width:1px
    style G fill:#ffb,stroke:#333,stroke-width:1px
```

Entity/Topology Relationships (graph):



Sources: Dynatrace documentation and OpenTelemetry specifications detail these enrichment steps. For example, Dynatrace's OneAgent "enriches telemetry at the source with primary fields and tags" [1†L23-L32], and OpenTelemetry resource detectors can automatically populate many of the above attributes. Each step above is rule-based and does **not** involve machine learning. [48†L25-L32] [55†L64-L68]